ARA

Autonomous Remediation Agent

Fleet management, autonomous incident remediation, and intelligent monitoring for multi-server Docker infrastructure. Detect, diagnose, and resolve production incidents without human intervention.

Version: 1.0

Documentation Date: September 2026

Platform: Linux / Docker / Multi-Node Fleet

Repository: github.com/Unlimited-demi/happy-galileo

Contents

01	Platform Overview	3
02	System Architecture	4
03	Core Features	5
04	Installation & Setup	7
05	Fleet Management	9
06	Monitoring & Detection	10
07	Incident Resolution Workflow	12
08	Command Reference	14
09	Tiered Remediation Engine	16
10	Dashboard & Telemetry	17
11	Troubleshooting	18
12	API Reference	19

1. Platform Overview

ARA is an autonomous AI-Ops platform designed for multi-server Docker infrastructure. It provides fleet-wide monitoring, intelligent anomaly detection, and autonomous incident remediation — closing the loop from error detection to code fix to production deployment without human intervention.

What It Does

FLEET MONITORING

Monitors all Docker containers across multiple servers. Collects CPU, memory, disk, and network metrics. Streams telemetry to a central dashboard.

ANOMALY DETECTION

Scans container logs with 17+ multi-language error patterns. Detects Python, Node.js, Go, Java, PHP/Laravel, Ruby stack traces, HTTP 5xx errors, OOM kills, and network failures.

AUTONOMOUS REMEDIATION

Dispatches OpenCode AI agent to fix detected errors. The agent checks out a branch, implements the fix, commits, and prepares for human review.

INCIDENT LIFECYCLE

Complete workflow: detect → classify → fix → verify → merge → deploy. Every incident has a comprehensive dossier with proof of resolution.

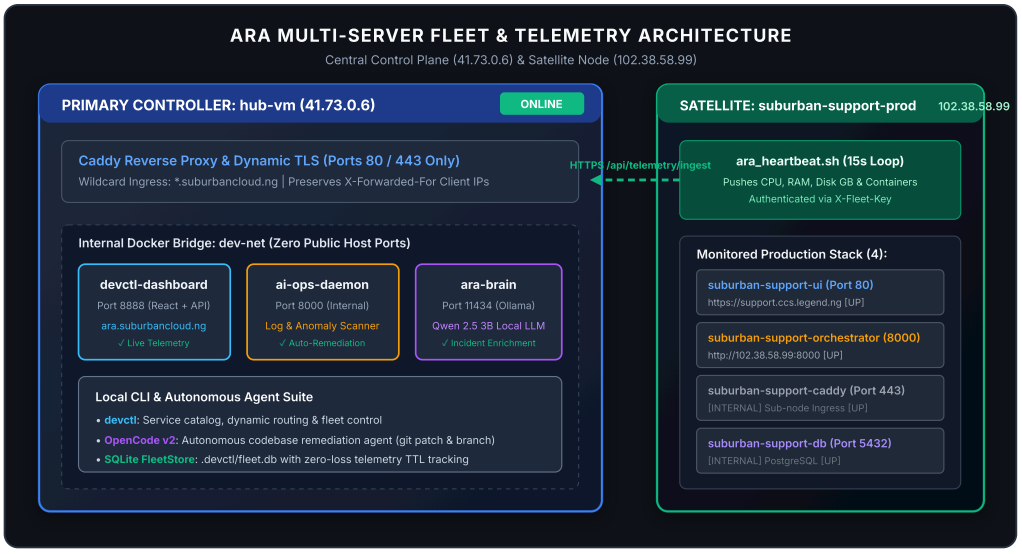
Design Principles

- **Outbound-Only Telemetry:** Sub-nodes push heartbeats to the hub. No inbound ports required. Commands piggyback on heartbeat responses.
- **Zero Public Ports:** All containers communicate over an internal Docker network (`dev-net`). Caddy handles TLS termination.
- **Safety First:** Config backups before writes, `nginx -t` validation before reload, rollback on failure, mail server detection to avoid breaking Mailcow.
- **Idempotent Operations:** Every script checks if work is already done before executing. Safe to run multiple times.

- **Language Agnostic:** Grammar-based error detection works across any programming language without hardcoding vendor names.

2. System Architecture

Hub-and-Spoke Model



Component Breakdown

COMPONENT	LOCATION	PURPOSE
`ai_ops/daemon.py`	Docker container	Monitoring loop: health checks, log scanning, incident creation, remediation dispatch
`ai_ops/classifier.py`	Python module	Grammar-based anomaly detection with 17+ multi-language error patterns
`ai_ops/remediation.py`	Python module	Tiered remediation engine (L0–L3) with auto-restart and network resync
`ai_ops/health_checker.py`	Python module	Container health inspection with direct HTTP probing on Docker network
`ai_ops/metrics.py`	Python module	Server and container metrics collection (CPU, memory, disk, network)

COMPONENT	LOCATION	PURPOSE
`ai_ops/telemetry.py`	Python module	Fleet telemetry streamer — pushes heartbeats to central hub every 15 seconds
`ai_ops/docker_socket.py`	Python module	Docker Engine API client with connection pooling and raw binary log parsing
`devctl/cli.py`	CLI tool	Unified CLI for service management, incident handling, fleet operations
`dashboard/`	React + Flask	Fleet dashboard with real-time node status, incidents, and service directory
`infra/security/`	Bash scripts	Proxy conflict resolver, UFW firewall hardening, Caddy TLS configuration

Data Flow

1. **Collection:** AI-Ops daemon on each node inspects containers via Docker socket, scans logs, collects metrics
2. **Detection:** Classifier analyzes logs for errors using multi-language grammar patterns
3. **Incident:** When error detected, incident created on Incident Bus with full context
4. **Remediation:** L1/L2 auto-remediation attempts (restart, network resync). If insufficient, dispatch to OpenCode
5. **Fix:** OpenCode checks out branch, implements fix, commits, pushes
6. **Telemetry:** Node streams heartbeat with services, incidents, and metrics to hub every 15s
7. **Dashboard:** Hub displays fleet-wide status, incidents, and node health in real-time

3. Core Features

3.1 Multi-Language Error Detection

The classifier uses grammar-based syntactic analysis to detect errors across any programming language:

LANGUAGE	PATTERNS DETECTED	EXAMPLES
PYTHON	Traceback, File/line frames, exception classes	`Traceback (most recent call last):`
NODE.JS	V8 runtime errors, unhandled rejections, module errors	`TypeError: Cannot read properties of undefined`
GO	Goroutine panics, fatal errors	`goroutine 1 [running]: panic: runtime error`
JAVA/JVM	Thread exceptions, caused-by chains, qualified exceptions	`Exception in thread "main" java.lang.NullPointerException`
PHP/LARAVEL	Fatal errors, parse errors, SQLSTATE, application errors	`[2024-01-15] local.ERROR: SQLSTATE[HY000]`
RUBY/RAILS	Error classes, 500 completions	`Completed 500 Internal Server Error in 234ms`

Additionally detects:

- **HTTP 5xx/429:** Upstream server outages and rate limits
- **Network failures:** ECONNREFUSED, ETIMEDOUT, ECONNRESET, ENOTFOUND
- **Process crashes:** OOMKilled, segmentation faults, fatal errors
- **Config errors:** Nginx/Caddy/Apache syntax errors, database auth failures

3.2 Incident Throttling & Contextual Analysis

Prevents incident spam while ensuring critical patterns get attention:

CONTEXT TYPE	TRIGGER	BEHAVIOR
TRANSIENT	Single occurrence	Create incident, normal severity
PERSISTENT	3+ same error type	Upgrade severity (warning→error, error→critical)
ESCALATING	5+ incidents in 5 min	Force critical severity, bypass throttle

Throttle rule: Maximum 1 incident per service per 5 minutes for the same error type.

3.3 Metrics Collection

SERVER METRICS

- CPU load average (1min, 5min, 15min) from `/proc/loadavg`
- Memory usage (total, used, available, percentage) from `/proc/meminfo`
- Disk usage per mount point (total, used, available, percentage)
- Network connections (TCP count via `ss`)
- System uptime from `/proc/uptime`

CONTAINER METRICS

- CPU percentage via `docker stats`
- Memory used / limit / percentage
- Network I/O (received / transmitted)
- Block I/O (read / written)
- Process count (PIDs)

Metrics are cached for 30 seconds and included in telemetry heartbeats to the central hub.

3.4 Reverse Proxy Detection

Smart detection of existing virtual hosts across all major web servers:

PROXY	LOCATIONS SCANNED	DIRECTIVES PARSED
Nginx	`sites-enabled/*`, `sites-available/*`, `conf.d/*.conf`, `nginx.conf`	`server_name`, `proxy_pass`

PROXY	LOCATIONS SCANNED	DIRECTIVES PARSED
Apache	`sites-enabled/*.conf`, `sites-available/*.conf`, `conf.d/*.conf`	`ServerName`, `ServerAlias`, `ProxyPass`
Caddy	`conf.d/*`, `Caddyfile`, `*.caddy`	Domain blocks, `reverse_proxy`
Traefik	Docker labels	`traefik.http.routers.*.rule`
Env Vars	Container environment	`VIRTUAL_HOST`, `APP_URL`, `*_DOMAIN`

Each nginx `server {}` block is parsed separately to correctly map `server_name` to its corresponding `proxy_pass` upstream — no cartesian product confusion.

3.5 Safety Features

- **Config backups:** Every nginx/Caddy config is backed up before modification
- **Validation:** `nginx -t` before reload, rollback on failure
- **Mail server detection:** Auto-skips UFW if Mailcow/Postfix/Dovecot detected
- **Docker port awareness:** UFW auto-allows Docker published ports
- **Idempotency:** All scripts check if work is already done
- **Vendor domain filtering:** Excludes `download.redis.io`, `github.com`, etc. from false-positive domain detection

4. Installation & Setup

4.1 Central Hub Setup (Primary Server)

The central hub runs the fleet dashboard and telemetry ingest endpoint. It receives heartbeats from all sub-nodes.

```
# Clone the repository
git clone https://github.com/Unlimited-demi/happy-galileo.git /opt/happy-ga
cd /opt/happy-galileo

# Configure environment
cp infra/.env.example infra/.env
# Edit infra/.env:
#   NODE_NAME=Primary Workstation
#   BASE_DOMAIN=suburbancloud.ng
#   CADDY_HTTP_PORT=80
#   CADDY_HTTPS_PORT=443
#   CENTRAL_HUB_URL=          (leave empty — this IS the hub)
#   FLEET_KEY=default-fleet-key

# Start the hub
cd infra
docker compose -f docker-compose.infra.yml --env-file .env up -d --build

# Discover local containers
devctl discover --force
```

> NOTE
>
> The hub's dashboard is accessible at `https://ara.suburbancloud.ng` via Caddy's on-demand TLS.

4.2 Sub-Node Enrollment

Sub-nodes run the AI-Ops daemon and stream telemetry to the hub. They can coexist with existing services (nginx, Mailcow, other Docker containers).

```
# On the sub-node server, run the enrollment script:
curl -sSL https://raw.githubusercontent.com/Unlimited-demi/happy-galileo/mas
  sudo bash -s -- \
    --node-name "vm2" \
    --domain "vm2.suburbancloud.ng" \
    --hub-url "https://ara.suburbancloud.ng/api/telemetry/ingest" \
    --github-token "ghp_your_token_here"
```

WHAT THE ENROLLMENT SCRIPT DOES:

1. **Prerequisites:** Installs git, curl, ufw, fail2ban, python3, jq
2. **Docker:** Installs Docker and Docker Compose if not present
3. **Codebase:** Clones the repository to `/opt/happy-galileo`
4. **Node.js:** Installs Node.js 20, Playwright, OpenCode CLI
5. **Docker network:** Creates `dev-net` internal network
6. **devctl CLI:** Installs Python venv and CLI wrapper at `/usr/local/bin/devctl`
7. **Proxy resolution:** Detects existing nginx/Caddy/Apache, configures ARA Caddy on internal port (8080+)
8. **Firewall:** Runs UFW hardening (skips if mail server detected)
9. **Infrastructure:** Builds and starts Caddy + AI-Ops daemon + Dashboard containers
10. **Discovery:** Auto-discovers existing Docker containers and registers them for monitoring

4.3 Environment Variables

VARIABLE	HUB	SUB-NODE	DESCRIPTION
`NODE_NAME`	Primary Workstation	vm2, vm3, vm4	Display name shown in fleet dashboard
`BASE_DOMAIN`	suburbancloud.ng	vm2.suburbancloud.ng	Domain scope for service URLs
`CADDY_HTTP_PORT`	80	8080+ (auto-detected)	HTTP port for Caddy
`CADDY_HTTPS_PORT`	443	8443	HTTPS port for Caddy
`CENTRAL_HUB_URL`	*(empty)*	https://status.../api/telemetry/ingest	Hub telemetry endpoint (empty = standalone)

VARIABLE	HUB	SUB-NODE	DESCRIPTION
`FLEET_KEY`	default-fleet-key	default-fleet-key	Shared secret for fleet authentication
`GITHUB_TOKEN`	ghp_...	ghp_...	GitHub PAT for OpenCode code access

5. Fleet Management

5.1 Node Name Normalization

To prevent duplicate nodes (e.g., `vm4` vs `vm-04`), all node names are automatically normalized:

- `suburban-support-prod (102.38.58.99)` → `vm2`
- `vm-03` → `vm3`
- `vm-04` → `vm4`
- Other names pass through unchanged

5.2 Fleet Commands

`devctl fleet list`

Display all nodes in the fleet with their status, last heartbeat time, container count, and service count.

```
devctl fleet list
```

Output example:

```
═══════════════════════════════════════════════════════════════════
📡 FLEET NODES
═══════════════════════════════════════════════════════════════════
NODE                 STATUS       LAST SEEN       CONTAINERS   SERVICES
─────────────────────────────────────────────────────────────────
vm2                  🟢 ONLINE    12s ago         27           27
vm3                  🟢 ONLINE    8s ago          15           15
vm4                  🟢 ONLINE    3s ago          19           19
primary              🟢 ONLINE    1s ago          6            6
```

`devctl fleet prune`

Remove stale nodes that haven't sent heartbeats within the specified time window.

```
devctl fleet prune [--max-age MINUTES] [--yes]
```

FLAG	DEFAULT	DESCRIPTION
`--max-age`	60	Maximum age in minutes before a node is considered stale
`--yes`	false	Skip confirmation prompt

5.3 Node States

STATUS	CONDITION	MEANING
🟢 **ONLINE**	Last seen < 60s	Node is actively streaming telemetry
🟡 **STALE**	Last seen 60s–5min	Node may be experiencing issues
🔴 **OFFLINE**	Last seen > 5min	Node has stopped reporting

6. Monitoring & Detection

6.1 Health Check Signals

The AI-Ops daemon evaluates 6 signals to determine container health:

#	SIGNAL	SOURCE	INDICATION
1	Running state	Docker inspect	Container is running vs stopped/exited
2	OOM kills	Docker inspect State.OOMKilled	Container was killed due to memory exhaustion
3	Restart count	Docker inspect RestartCount	Container is crash-looping
4	Docker HEALTHCHECK	Docker inspect State.Health.Status	Application-level health check result
5	HTTP probe	Direct socket to container IP:port	HTTP 5xx responses indicate application failure
6	Log scanning	Docker logs API + classifier	Error patterns, stack traces, exceptions

6.2 HTTP Probing

Even when Docker reports a container as "running", the application inside may be returning 502/503 errors. The health checker resolves the container's IP on the Docker network and sends a direct HTTP request:

```
Container: aiops-grafana
  Docker IP: 172.18.0.5
  Port: 3000
  Probe: GET http://172.18.0.5:3000/ → HTTP 502 Bad Gateway
  Result: UNHEALTHY (HTTP 502 from aiops-grafana:3000)
```

Containers in the skip list (`ai-ops-daemon` , `caddy`) are not probed to avoid feedback loops.

6.3 Docker Log Parsing

Docker's multiplexed log streams contain 8-byte binary headers per frame:

```
[stream_type(1), 0, 0, 0, size(4 big-endian)] + frame_data
```

The log parser reads raw bytes, parses frame headers, extracts frame data, and decodes UTF-8 — all before pattern matching. This prevents corruption from binary headers that would otherwise appear as replacement characters (`\\xef\\xbf\\xbd`).

6.4 Monitoring Cycle

```
Every 15 seconds:
  1. List all registered services
  2. For each service:
     a. Inspect container (running, OOM, restarts, healthcheck)
     b. Probe HTTP (direct socket to container IP:port)
     c. Fetch logs (Docker API, last 100 lines)
     d. Classify logs (17+ error patterns)
     e. Determine health status
     f. If unhealthy:
        - L1: Auto-restart (max 3 attempts)
        - L2: Network resync + proxy reload
        - L3: Dispatch to OpenCode
     g. Create incident if error detected
  3. Collect server & container metrics
  4. Stream telemetry to hub (if sub-node)
```

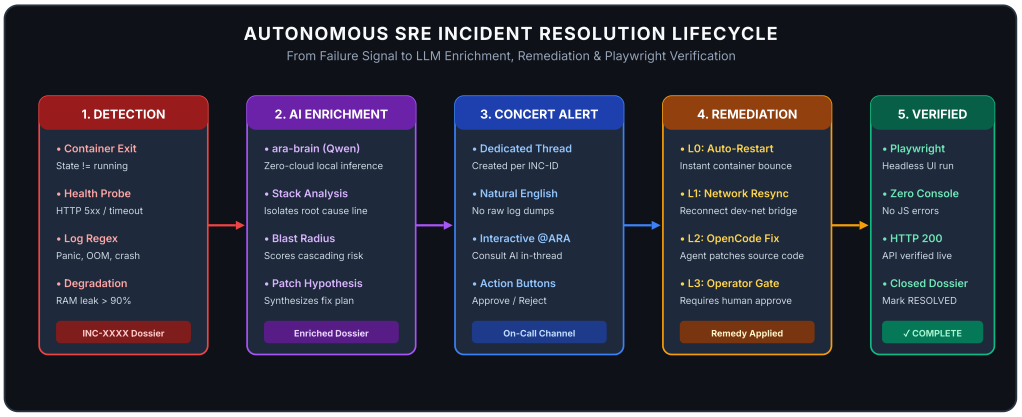
6.5 Custom Error Patterns

Operators can extend detection without code changes by adding regex patterns to:

```
~/.devctl/error_patterns.txt    # Lines matching these are flagged as errors
~/.devctl/noise_patterns.txt    # Lines matching these are ignored
```

Files are re-read every 60 seconds. One regex per line, `#` for comments.

7. Incident Resolution Workflow



The complete incident lifecycle from detection to production deployment:

01 Detect

AI-Ops daemon scans container logs and detects an error pattern. Creates an incident on the Incident Bus with full context: error snippet, container name, service, severity, and contextual classification (transient/persistent/escalating).

02 Classify & Throttle

Incident throttle checks if this is a duplicate (same service, same error, within 5 minutes). If throttled, the incident is suppressed. If persistent or escalating, severity is upgraded.

03 Auto-Remediate (L1/L2)

If the container is stopped or returning HTTP 5xx, L1 auto-restarts it (max 3 attempts). If the container lost network connectivity, L2 resyncs the Docker network and reloads the proxy.

04 Dispatch to OpenCode (L3)

If auto-remediation is insufficient, the incident is dispatched to the OpenCode AI agent. A comprehensive dossier is generated with error details, logs, workspace path, and instructions.

05 Fix

OpenCode checks out a fix branch (`fix/INC-20240901-123456-service`), analyzes the code, implements the fix, runs tests, commits, and pushes to the remote.

06 Verify

The fix is deployed locally. DevOps reviews the incident dossier, checks the code changes, and verifies the fix resolves the issue.

07 Merge

DevOps runs `devctl merge INC-xxx` . The fix branch is merged to master with `--no-ff` , pushed to remote, and the branch is deleted. The dossier is updated with merge commit and timestamp.

08 Deploy

DevOps runs `devctl deploy <service>` (or uses `--deploy` flag with merge). The service is rebuilt and restarted with the merged code. The incident is marked RESOLVED.

Incident Dossier Structure

Each incident generates a comprehensive markdown file at `~/.devctl/incidents/INC-xxx.md` :

```
# Incident Dossier: INC-20240901-123456

## Summary
- Service: chaos-api
- Container: chaos-api
- Severity: CRITICAL
- Context: escalating
- Error Category: Node.js Runtime Error
- Detected: 2024-09-01 12:34:56 UTC
- Status: ✅ RESOLVED & MERGED

## Error Details
### Error Snippet
TypeError: Cannot read properties of undefined (reading 'id')

### Full Logs (Last 50 Lines)
[application logs here]

## Resolution
### Fix Branch
```

```
- Branch: fix/INC-20240901-123456-chaos-api
- Commit: a1b2c3d4e5f6
- Message: Fix null pointer in user handler
- Files Changed: 2
  - src/handlers/user.js
  - tests/user.test.js

### Code Changes (Diff Summary)
[diff output]

### Verification Results
- Status: ✅ PASSED
- Deployment: Running
- Test Results: [test output]

## Proof of Resolution
### Merge Information
- Merged By: admin
- Merge Commit: f6e5d4c3b2a1
- Merged At: 2024-09-01 13:45:00 UTC
- Production Deployed: ✅ Yes
```

8. Command Reference

Service Management

`devctl expose <service> [port]`

Expose a container service through Caddy with dynamic HTTPS. Creates a wildcard TLS certificate via Let's Encrypt.

```
devctl expose grafana 3000 devctl expose api --domain api.example.com
8000
```

`devctl hide <service>`

Remove the public HTTPS route for a service. The container remains running and monitored internally.

```
devctl hide grafana
```

`devctl list`

List all registered services with their public URLs, container status, and health.

```
devctl list
```

`devctl logs <service> [--tail N]`

View container logs for a service. Parses Docker's multiplexed log streams correctly.

```
devctl logs chaos-api --tail 50
```

Discovery & Indexing

`devctl discover [--force]`

Auto-discover running Docker containers. Detects existing nginx/Caddy/Apache virtual hosts, workspace paths, and classifies containers by role.

```
devctl discover --force
```

devctl reindex

Flush cached state and re-index all containers. Equivalent to `devctl discover --force`.

```
devctl reindex
```

Incident Management

devctl incident [list|inspect|claim|resolve|purge]

Manage the incident bus. List open incidents, inspect details, claim for resolution, or purge all.

```
devctl incident list devctl incident inspect INC-20240901-123456 devctl incident purge
```

devctl dispatch [incident_id]

Dispatch an incident to OpenCode for autonomous remediation. Launches in tmux session if available.

```
devctl dispatch INC-20240901-123456 devctl dispatch --dry-run
```

devctl merge <incident_id> [--deploy]

Merge a verified fix branch back to master. Shows commits, asks for confirmation, pushes, and optionally deploys.

```
devctl merge INC-20240901-123456 devctl merge INC-20240901-123456 --deploy
```

Deployment

devctl deploy [service] [--pull] [--no-cache]

Rebuild and restart containers. Optionally pull latest code first.

```
devctl deploy chaos-api devctl deploy all --pull devctl deploy --no-cache api
```

Fleet Management

devctl fleet [list|prune]

List all fleet nodes or prune stale ones.

```
devctl fleet list devctl fleet prune --max-age 30 --yes
```

Telemetry & Diagnostics

devctl heartbeat [--hub URL] [--name NODE]

Send and test telemetry heartbeat to the central hub.

```
devctl heartbeat --hub https://ara.suburbancloud.ng/api/telemetry/ingest
```

devctl test <target>

Run Playwright visual and diagnostic tests against a service URL.

```
devctl test chaos-api devctl test https://ara.suburbancloud.ng
```

devctl doctor

Check system health: Docker access, devctl state, Caddy connectivity, OpenCode availability.

```
devctl doctor
```

9. Tiered Remediation Engine

The remediation engine operates in 4 escalating tiers:

L0 — Observe

Trigger	Log anomaly detected, container healthy
Action	Create incident, record metrics, no automatic action
Use Case	Non-critical warnings, informational errors

L1 — Auto-Restart

Trigger	Container stopped OR HTTP 5xx response
Action	Restart container (max 3 attempts per incident window)
Use Case	OOM kills, crash loops, 502/503 errors
Guard	After 3 failed restarts, escalate to L2

L2 — Network Resync

Trigger	Container lost Docker network connectivity
Action	Disconnect and reconnect to `dev-net`, reload proxy (Caddy/nginx/Apache)
Use Case	Container unreachable on Docker network despite running
Guard	If resync fails, escalate to L3

L3 — OpenCode Escalation

Trigger	L1/L2 insufficient, code-level fix required
Action	Generate incident dossier, dispatch to OpenCode AI agent

Use Case	Application bugs, null pointer exceptions, logic errors
Guard	OpenCode works autonomously in tmux, creates fix branch

SAFETY

Auto-remediation can be disabled by setting `AUTO_REMEDIATION_ENABLED=false` in the environment. When disabled, incidents are created but no automatic restart or network resync is attempted.

10. Dashboard & Telemetry

10.1 Fleet Dashboard

The dashboard is a React application served by the hub at `https://status.BASE_DOMAIN`. It displays:

- **KPI Metrics:** Total nodes, containers, services, open incidents, server load
- **Fleet Matrix:** All nodes with status (ONLINE/STALE/OFFLINE), container count, service count
- **Service Directory:** Per-node service listing with container name, archetype, port, health, public URL
- **Incident Console:** Open and resolved incidents with severity, context badges, dispatch/merge actions
- **Health Digest:** Summary of fleet health trends and degradation patterns

10.2 Telemetry Payload

Each node streams a heartbeat every 15 seconds containing:


```
{
  "node_name": "vm4",
  "base_domain": "vm4.suburbancloud.ng",
  "timestamp": 1725192000.0,
  "status": "ONLINE",
  "containers_count": 19,
  "services_count": 19,
  "services": [
    {
      "service_name": "aiops-grafana",
      "container_name": "aiops-grafana",
      "port": 3000,
      "url": "https://aideveops.suburban.ng",
      "container_type": "web",
      "image": "grafana/grafana:10.2.2",
      "version": "10.2.2",
      "container_status": "Up 3 days"
    }
  ],
  "open_incidents_count": 2,
  "open_incidents": [...],
```

```
  "metrics": {
    "server": {
      "load_average": {"1min": 0.45, "5min": 0.62, "15min": 0.58},
      "memory": {"total_bytes": 16777216000, "used_bytes": 8388608000, "usa
      "disks": [...],
      "network": {"tcp_connections": 142},
      "uptime_seconds": 864000
    },
    "containers": [
      {
        "name": "aiops-grafana",
        "cpu_percent": 2.3,
        "memory_used_bytes": 268435456,
        "memory_limit_bytes": 1073741824,
        "memory_percent": 25.0,
        "network_io": "1.2GB / 800MB",
        "block_io": "50MB / 12MB",
        "pids": 12
      }
    ]
  },
  "trends": {}
}
```


10.3 Command Channel

The hub can send commands back to nodes via heartbeat responses. Only whitelisted commands are accepted:

COMMAND	ACTION
`restart_container`	Restart a specific container
`resync_network`	Disconnect and reconnect container to dev-net, reload proxy
`purge_incidents`	Clear all incident files on the node

> **SECURITY**
>
> Arbitrary shell commands are NOT accepted. Only the three whitelisted commands above can be executed remotely. This prevents unauthorized command execution even if the hub is compromised.

11. Troubleshooting

Common Issues

NODE APPEARS TWICE IN DASHBOARD (E.G., VM4 AND VM-04)

Cause: Inconsistent node naming across enrollment and heartbeat.

Fix: Update all nodes to latest code which normalizes names. Then prune duplicates:

```
cd /opt/happy-galileo && git reset --hard origin/master
devctl fleet prune --yes
```

SERVICES SHOW [INTERNAL - NO PUBLIC URL] DESPITE HAVING NGINX VIRTUAL HOSTS

Cause: Domain detection not finding the nginx configs.

Fix: Re-run discovery with force:

```
devctl discover --force
```

If still not detected, check that nginx configs are in standard locations (`/etc/nginx/sites-enabled/`, `/etc/nginx/conf.d/`, `/etc/nginx/sites-available/`).

REDIS/DATABASE CONTAINERS SHOWING VENDOR URLS (DOWNLOAD.REDIS.IO)

Cause: Environment variables like `REDIS_DOWNLOAD_URL` being picked up as service domains.

Fix: Updated code filters known vendor domains. Update and re-discover:

```
cd /opt/happy-galileo && git reset --hard origin/master
devctl discover --force
```

UFW BLOCKED MAIL PORTS ON MAILCOW SERVER

Cause: Older version of UFW script didn't detect mail servers.

Fix: Updated code auto-skips UFW when Mailcow/Postfix/Dovecot detected. If UFW already broke mail:

```
ufw allow 25/tcp    # SMTP
ufw allow 587/tcp   # Submission
ufw allow 993/tcp   # IMAPS
ufw allow 143/tcp   # IMAP
ufw allow 110/tcp   # POP3
ufw allow 995/tcp   # POP3S
```

OPENCODE DOESN'T MERGE FIX BRANCH AFTER RESOLUTION

Cause: By design — merge requires human confirmation.

Fix: After verifying the fix, run:

```
devctl merge INC-20240901-123456 --deploy
```

DASHBOARD SHOWS STALE NODES THAT NO LONGER EXIST

Cause: Old node state persisted on disk.

Fix: Prune stale nodes:

```
devctl fleet prune --max-age 5 --yes
```

Diagnostic Commands

```
# Check system health
devctl doctor

# View AI-Ops daemon logs
docker logs ai-ops-daemon --tail 50

# Check node telemetry
devctl heartbeat

# View container health
devctl list

# Inspect specific incident
devctl incident inspect INC-xxx

# View container logs
devctl logs chaos-api --tail 100
```

```
# Check fleet status
devctl fleet list
```

12. API Reference

Dashboard API Endpoints

METHOD	ENDPOINT	DESCRIPTION
GET	`/api/status`	Current node status: services, containers, incidents
GET	`/api/fleet/nodes`	All fleet nodes with telemetry data
GET	`/api/incidents?all=true`	List incidents (open only by default, `?all=true` includes resolved)
POST	`/api/incidents/{id}/dispatch`	Dispatch incident to OpenCode
POST	`/api/incidents/purge`	Purge all incident files
POST	`/api/telemetry/ingest`	Receive heartbeat from sub-nodes (hub only)

Telemetry Ingest Payload

Sub-nodes POST to `/api/telemetry/ingest` with header `X-Fleet-Key: <key>`:

```
POST /api/telemetry/ingest
Content-Type: application/json
X-Fleet-Key: default-fleet-key

{
  "node_name": "vm4",
  "base_domain": "vm4.suburbancloud.ng",
  "timestamp": 1725192000.0,
  "status": "ONLINE",
  "containers_count": 19,
  "services_count": 19,
  "services": [...],
  "open_incidents_count": 2,
  "open_incidents": [...],
  "metrics": {
    "server": { ... },
    "containers": [ ... ]
```

```
  }
}
```

Response:

```
{
  "status": "accepted",
  "node": "vm4",
  "commands": []   // Optional: hub-issued commands to execute
}
```

File Structure

```
/opt/happy-galileo/
├── ai_ops/
│   ├── daemon.py              # Main monitoring loop
│   ├── classifier.py          # Multi-language error detection
│   ├── remediation.py         # L0-L3 tiered remediation
│   ├── health_checker.py      # Container health inspection
│   ├── metrics.py             # Server & container metrics
│   ├── telemetry.py           # Fleet telemetry streamer
│   ├── docker_socket.py       # Docker Engine API client
│   ├── incident_throttle.py   # Incident rate limiting
│   └── codebase_resolver.py   # Git workspace detection
├── devctl/
│   ├── cli.py                 # CLI entry point
│   ├── core/
│   │   ├── config.py          # Configuration
│   │   ├── caddy.py           # Caddy API manager
│   │   ├── docker_mgr.py      # Docker management
│   │   ├── domains.py         # Domain registry & detection
│   │   ├── incident_bus.py    # Incident state management
│   │   ├── fleet_state.py     # Fleet node tracking
│   │   └── incident_dossier.py # Dossier generation
│   └── commands/
│       └── fleet.py           # Fleet management commands
├── dashboard/
│   ├── app.py                 # Flask API server
│   ├── frontend/              # React application
│   └── static/                # Legacy static dashboard
├── infra/
│   ├── docker-compose.infra.yml
│   ├── .env.example
│   └── security/
│       ├── proxy_resolver.sh  # Proxy conflict resolution
│       └── ufw_setup.sh       # Firewall hardening
├── scripts/
│   └── setup-node.sh          # Sub-node enrollment script
├── services/
│   └── chaos-api/             # Test application
│       ├── stress-test.sh     # Fault injection test
```

```
│       └── test-l1-l2.sh     # L1/L2 escalation test
└── requirements.txt
```
